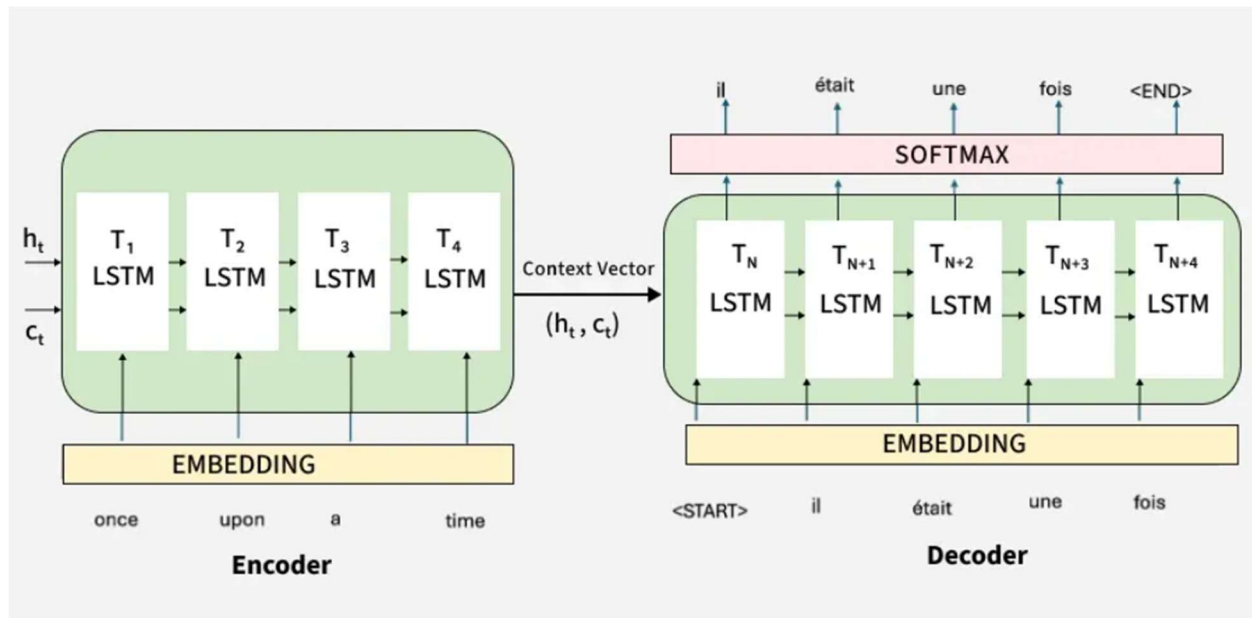




Encoder-Decoder Model

There are three main blocks in the encoder-decoder model,

- Encoder
- Hidden Vector
- Decoder

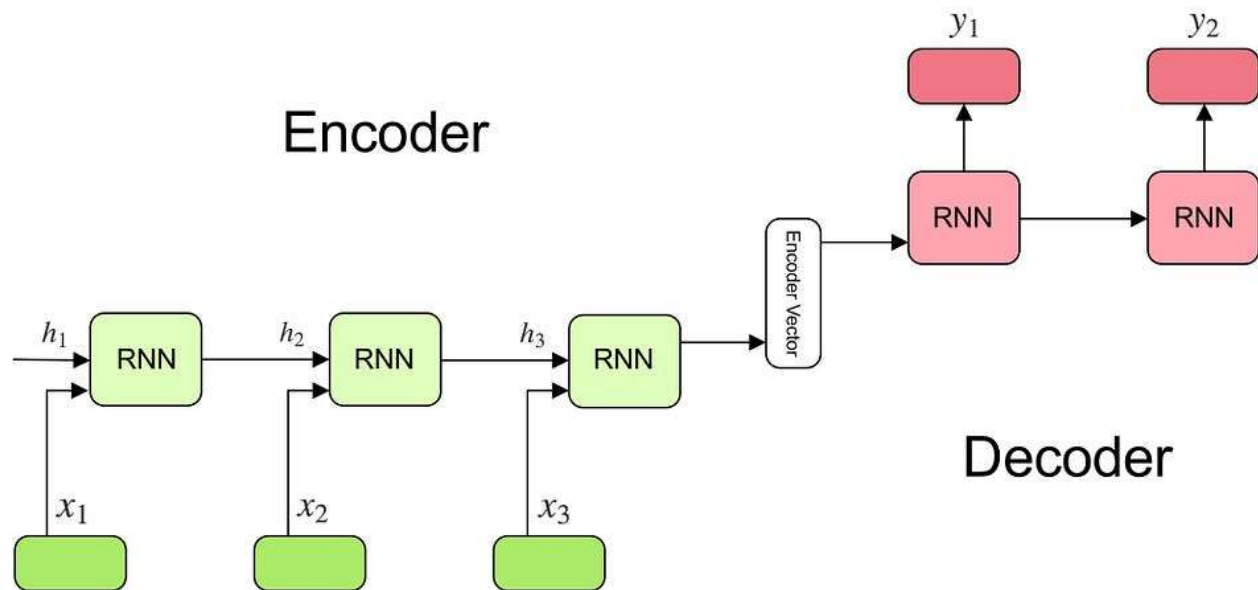
The Encoder will convert the input sequence into a single-dimensional vector (hidden vector). The decoder will convert the hidden vector into the output sequence.

Encoder-Decoder models are jointly trained to maximize the conditional probabilities of the target sequence given the input sequence.

How the Sequence to Sequence Model works?

In order to fully understand the model's underlying logic, we will go over the below illustration:

Press enter or click to view image in full size

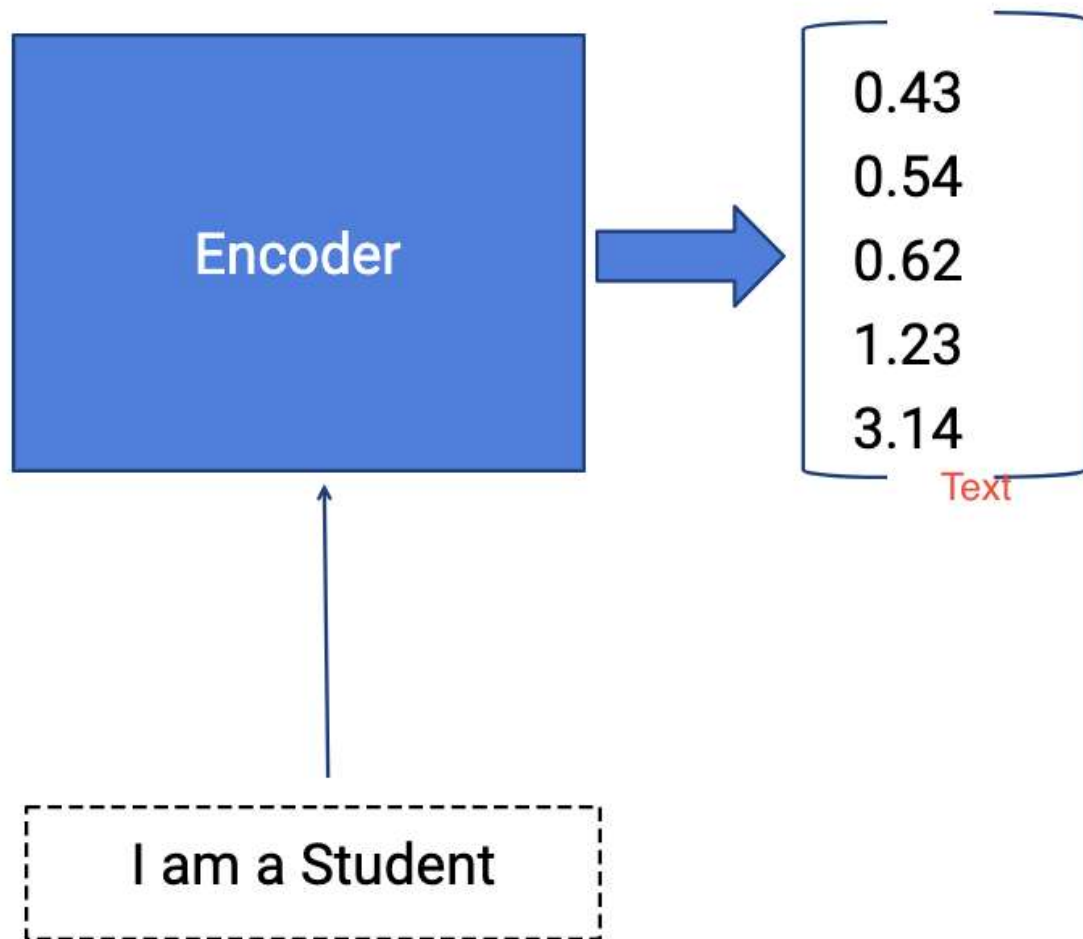


Encoder-decoder sequence to sequence model

Encoder

- Multiple RNN cells can be stacked together to form the encoder. RNN reads each inputs sequentially
- For every timestep (each input) t , the hidden state (hidden vector) h is updated according to the input at that timestep $X[i]$.
- After all the inputs are read by encoder model, the final hidden state of the model represents the context/summary of the whole input sequence.
- Example: Consider the input sequence "I am a Student" to be encoded. There will be totally 4 timesteps (4 tokens) for the Encoder model. At each time step, the hidden state h will be updated using the previous hidden state and the current input.

Press enter or click to view image in full size



Example: Encoder

- At the first timestep t_1 , the previous hidden state h_0 will be considered as zero or randomly chosen. So the first RNN cell will update the current hidden state with the first input and h_0 . Each layer outputs two things — updated hidden state and the output for each stage. The outputs at each stage are rejected and only the hidden states will be propagated to the next layer.
- The hidden states h_j are computed using the formula:

Press enter or click to view image in full size

$$h_t = f(W^{(hh)}h_{t-1} + W^{(hx)}x_t)$$

- At second timestep t2, the hidden state h1 and the second input X[2] will be given as input, and the hidden state h2 will be updated according to both inputs. Then the hidden state h1 will be updated with the new input and will produce the hidden state h2. This happens for all the four stages wrt example taken.
- A stack of several recurrent units (LSTM or GRU cells for better performance) where each accepts a single element of the input sequence, collects information for that element, and propagates it forward.
- In the question-answering problem, the input sequence is a collection of all words from the question. Each word is represented as x_i where i is the order of that word.

This simple formula represents the result of an ordinary recurrent neural network. As you can see, we just apply the appropriate weights to the previously hidden state $h_{(t-1)}$ and the input vector x_t .

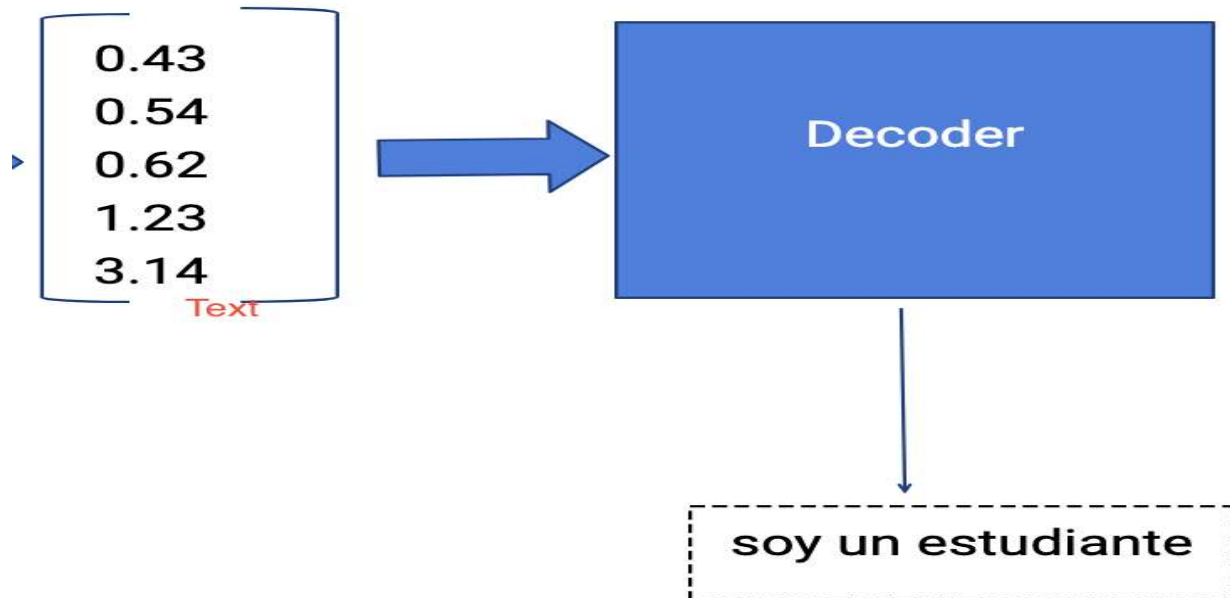
Encoder Vector

- This is the final hidden state produced from the encoder part of the model. It is calculated using the formula above.
- This vector aims to encapsulate the information for all input elements in order to help the decoder make accurate predictions.
- It acts as the initial hidden state of the decoder part of the model.

Decoder

- The Decoder generates the output sequence by predicting the next output Y_t given the hidden state h_t .
- The input for the decoder is the final hidden vector obtained at the end of encoder model.
- Each layer will have three inputs, hidden vector from previous layer h_{t-1} and the previous layer output y_{t-1} , original hidden vector h .
- At the first layer, the output vector of encoder and the random symbol START, empty hidden state h_{t-1} will be given as input, the outputs obtained will be y_1 and updated hidden state h_1 (the information of the output will be subtracted from the hidden vector).

- The second layer will have the updated hidden state h_1 and the previous output y_1 and original hidden vector h as current inputs, produces the hidden vector h_2 and output y_2 .
- The outputs occurred at each timestep of decoder is the actual output. The model will predict the output until the END symbol occurs.
- A stack of several recurrent units where each predicts an output y_t at a time step t .
- Each recurrent unit accepts a hidden state from the previous unit and produces an output as well as its own hidden state.
- In the question-answering problem, the output sequence is a collection of all words from the answer. Each word is represented as y_i where i is the order of that word.



Example: Decoder.

- Any hidden state h_i is computed using the formula:

Press enter or click to view image in full size

$$h_t = f(W^{(hh)} h_{t-1})$$

As you can see, we are just using the previous hidden state to compute the next one.

Output Layer

- We use Softmax activation function at the output layer.
- It is used to produce the probability distribution from a vector of values with the target class of high probability.
- The output y_t at time step t is computed using the formula:

Press enter or click to view image in full size

$$y_t = \text{softmax}(W^S h_t)$$

We calculate the outputs using the hidden state at the current time step together with the respective weight $W(S)$. Softmax is used to create a probability vector that will help us determine the final output (e.g. word in the question-answering problem).

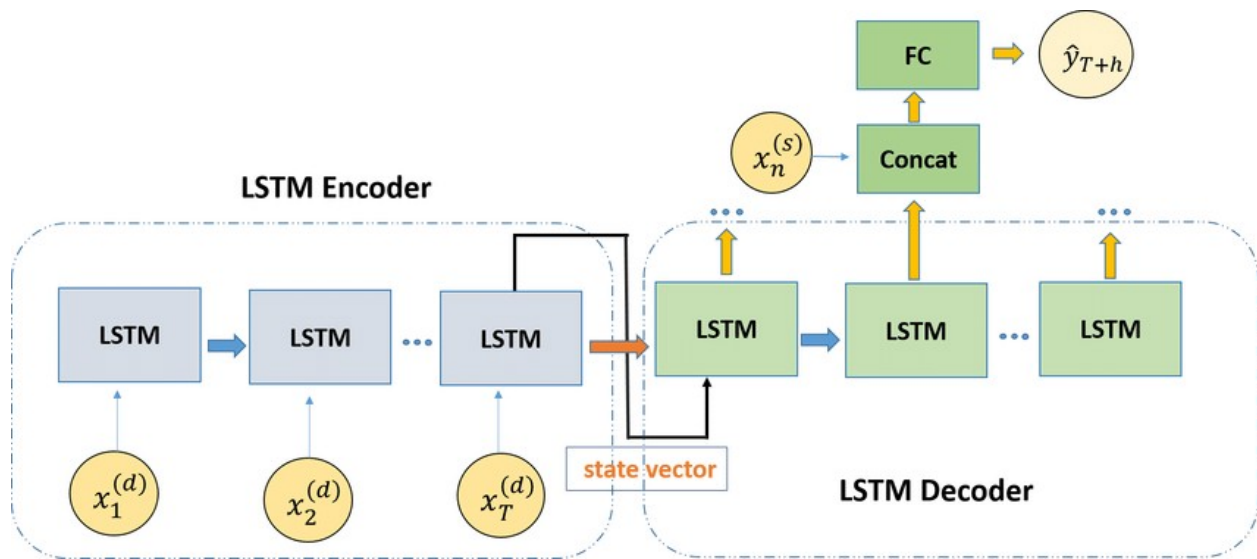
The power of this model lies in the fact that it can map sequences of different lengths to each other. As you can see the inputs and outputs are not correlated and their lengths can differ. This opens a whole new range of problems that can now be solved using such architecture.

Applications

It possesses many applications such as

- Google's Machine Translation
- Question answering chatbots
- Speech recognition
- Time Series Application etc.,

1. What this diagram represents (Big Picture)



This model performs **sequence-to-sequence translation**:

English input

once upon a time



French output

il était une fois

It does this using **two LSTMs**:

- Encoder LSTM → understands the input sentence
- Decoder LSTM → generates the output sentence

2. Encoder Workflow (Left Side)

Step 1: Input words → Embeddings

Input sentence:

once | upon | a | time

Each word is converted into a **vector** using an embedding layer.

So the encoder actually receives:

e_once, e_upon, e_a, e_time

Step 2: LSTM processes words one by one

The encoder LSTM runs **sequentially**:

Time step Input Hidden State

T1 once h_1

T2 upon h_2

T3 a h_3

T4 time h_4

At every step:

$$(h_t, c_t) = \text{LSTM}(x_t, h_{t-1}, c_{t-1})$$

Step 3: Final encoder state = Context Vector

After the **last word** (time):

- Final hidden state: h_4
- Final cell state: c_4

These together form the **context vector**:

$$\text{Context Vector} = (h_4, c_4)$$

Important

This context vector is the **entire meaning of the sentence** as far as the model is concerned.

3. Decoder Workflow (Right Side)

Now the decoder starts.

Step 4: Decoder initialization

The decoder LSTM is **initialized** with the encoder's final state:

$$h_0^{dec} = h_4^{enc}, c_0^{dec} = c_4^{enc}$$

This transfers the sentence meaning from encoder → decoder.

Step 5: Decoder input = shifted right outputs

Decoder input tokens:

<START> | il | était | une | fois

Why <START>?

- Decoder needs a starting signal
 - It predicts the **next word** at each step
-

Step 6: Decoder generates words step by step

At each time step:

Decoder Input Decoder Predicts

<START>	il
il	était
était	une
une	fois
fois	<END>

Internally, at step t:

$$(h_t, c_t) = \text{LSTM}(y_{t-1}, h_{t-1}, c_{t-1})$$

Step 7: Softmax produces probabilities

Each decoder LSTM output goes through:

Linear → Softmax

This produces a **probability distribution over the vocabulary**.

Example:

$P(il)=0.72$, $P(le)=0.15$, $P(un)=0.04$, ...

The word with the highest probability is selected.

Step 8: Stop condition

When the decoder predicts:

<END>

the generation stops.

4. Why this model works (Conceptually)

- Encoder **compresses meaning**
 - Decoder **expands meaning into a sentence**
 - LSTM memory preserves order and context
 - Softmax chooses the most likely word
-

5. Critical Limitation of THIS Architecture

✖ This model uses **only one context vector**.

That means:

- Long sentences → information loss
- Early words fade
- No word-to-word alignment

👉 This exact problem led to the invention of ATTENTION

6. One-Sentence Workflow Summary (Exam-Ready)

The encoder LSTM converts the input sequence into a fixed context vector, which initializes the decoder LSTM that generates the output sequence one token at a time using softmax predictions.

1. What is Teacher Forcing (Plain Definition)

Teacher forcing is a training technique where the decoder is fed the *correct previous target word* instead of its own prediction.

In short:

- **Training** → use ground truth
 - **Inference** → use model predictions
-

2. Where Teacher Forcing Appears in Your Diagram

Look at the decoder input row in your image:

<START> il était une fois

These are **not predicted words**.

They are the **true target sentence**, shifted right.

This is **teacher forcing**.

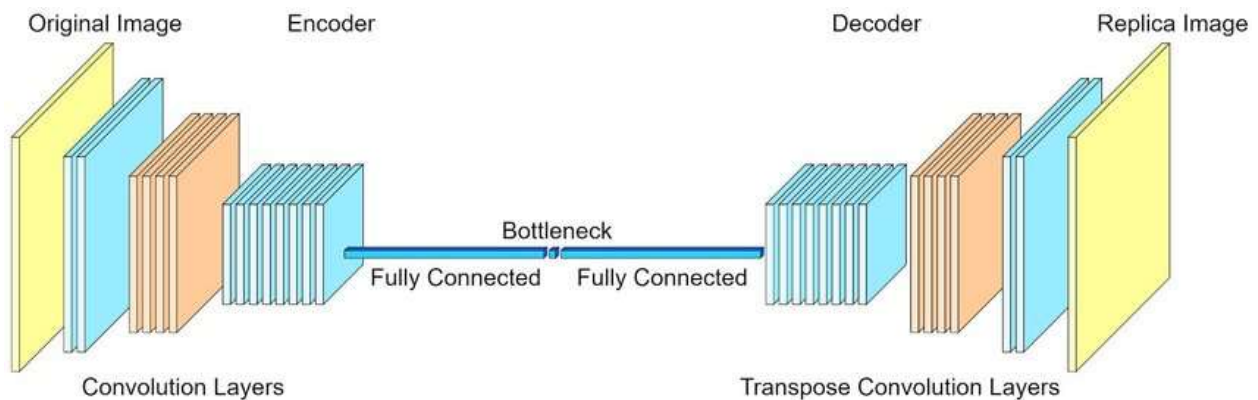
Teacher forcing is a training technique where the decoder is fed the true previous target token instead of its own prediction to stabilize and accelerate sequence learning.

Why Attention Mechanism Is Used (Core Reason)

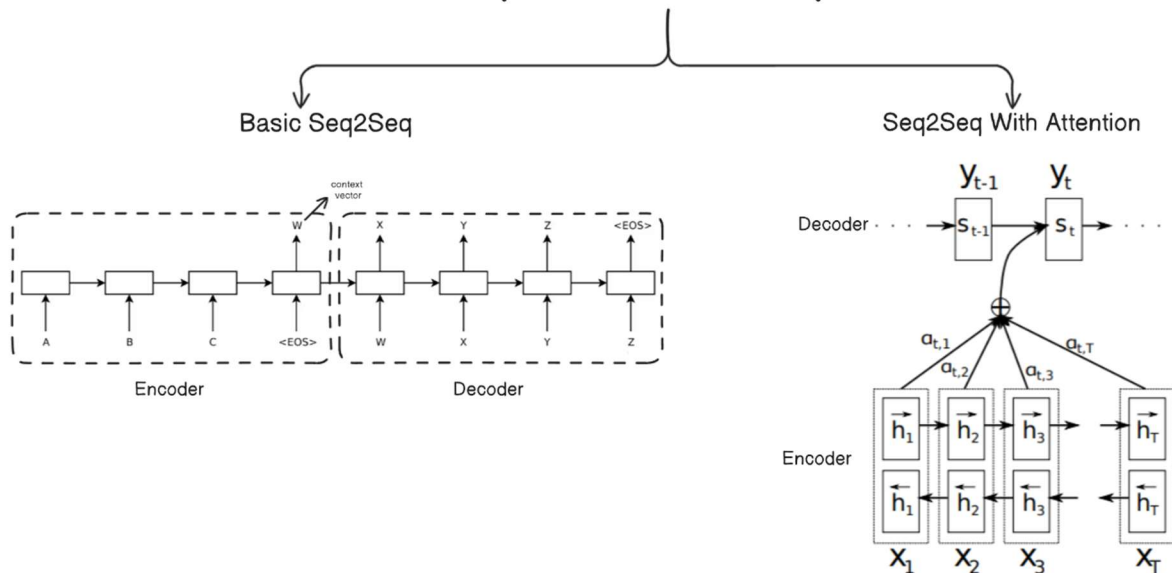
Attention is used so the model can dynamically focus on the most relevant parts of the input while producing each output.

This single sentence solves **multiple fundamental problems** in deep learning sequence models.

1. The Problem Before Attention (Very Important)



Sequence To Sequence



Classic Encoder-Decoder (LSTM/GRU) Problem

Before attention:

- Encoder reads the entire input sequence

- Compresses everything into **one fixed-size context vector**
- Decoder must generate the entire output using only this vector

Why this is bad

Imagine compressing:

once upon a time in a distant land there lived a king ...

into **one vector**.

This causes:

1. Information loss
2. Early words get forgotten
3. Long sentences perform poorly

This is called the **context vector bottleneck**.

2. No Word-to-Word Alignment

Without attention:

- Decoder has no idea which input word relates to the current output word

Example (translation):

Input: once upon a time

Output: il était une fois

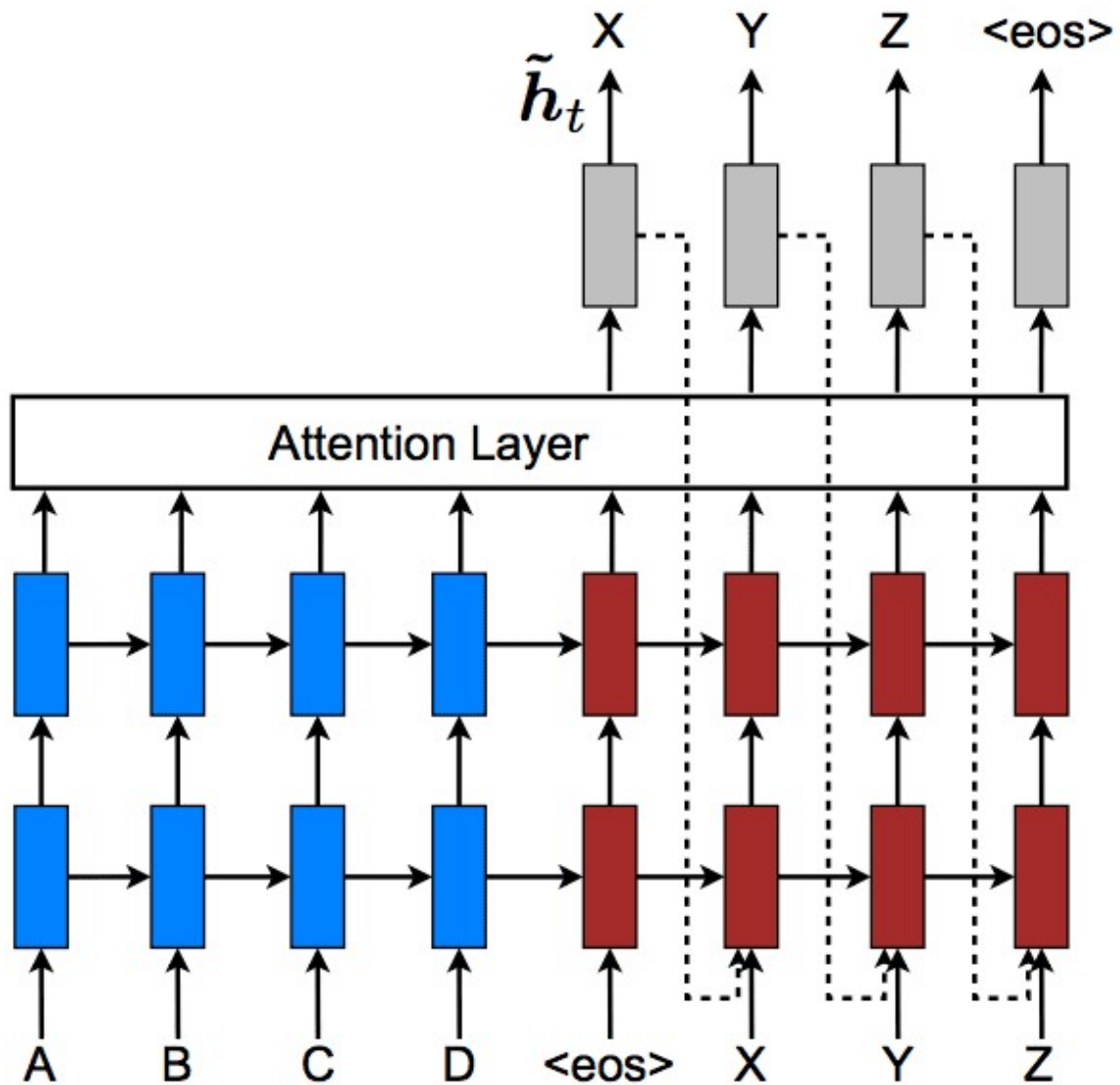
Without attention:

- Decoder guesses blindly using the context vector
- No explicit mapping like:
 - once → il
 - upon → était

This causes poor translations.

3. Attention Solves These Problems

[illegible]



What attention does differently

Instead of **one context vector**, attention allows:

- Access to **all encoder hidden states**
- At **every decoding step**

So for each output word, the model asks:

“Which input words should I focus on right now?”

4. Step-by-Step: What Attention Enables

Without Attention

Encoder → single vector → Decoder

With Attention

Encoder → all hidden states → Decoder (focuses dynamically)

At each step:

1. Compare decoder state with all encoder states
 2. Assign importance scores
 3. Create a weighted context
 4. Predict output
-

5. Human Analogy (Very Important)

When humans translate:

- We don't memorize the entire sentence at once
- We **look back** at the relevant words

Attention gives models this ability.

6. Why Attention Improves Performance

1. Handles Long Sequences

- No information compression into one vector
- Every word remains accessible

2. Improves Accuracy

- Better alignment
- More meaningful outputs

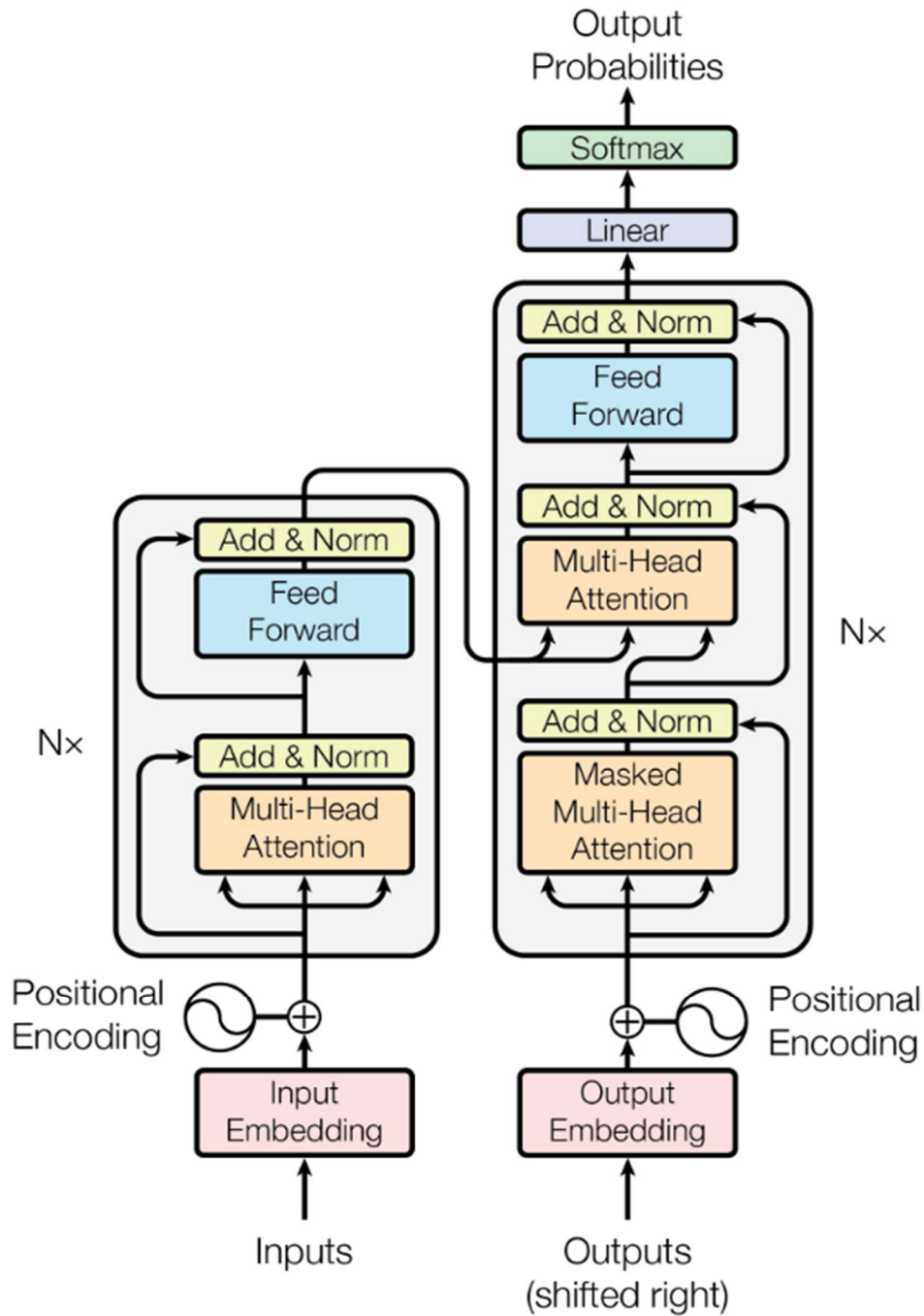
3. Enables Interpretability

- Attention maps show what the model focused on

4. Enables Parallelization (Transformers)

- Removes recurrence
- Makes large-scale training possible

7. Attention Is the Foundation of Transformers



The key insight of Transformers:

We don't need RNNs or CNNs. Attention alone is enough.

Without attention:

- Transformers do not work
- Models like BERT, GPT, T5 do not exist

Faults in encoder decoder architecture

1. Encoder decoder architecture did not perform well for sentences whose word count was greater than 25.
2. Also from the decoder side, to print any word, it does not require the entire sentence information, it only requires a specific word or a set of specific words. For example to translate the sentence "Turn off the lights " to hindi, the answer would be "light band karo". Over here for the word "band" only the words "turn" and "off" were important and not the entire sentence
3. But in the encoder decoder architecture, to output any word from the decoder, it required the entire input summary or the entire input sentence (which was not necessary). The decoder takes the entire input summary (context vector) for every output word in the encoder decoder architecture.
4. And this context vector was static, it would have been better if it would have been a dynamic representation of the input.

To address these issues in the encoder decoder architecture, attention mechanism was introduced

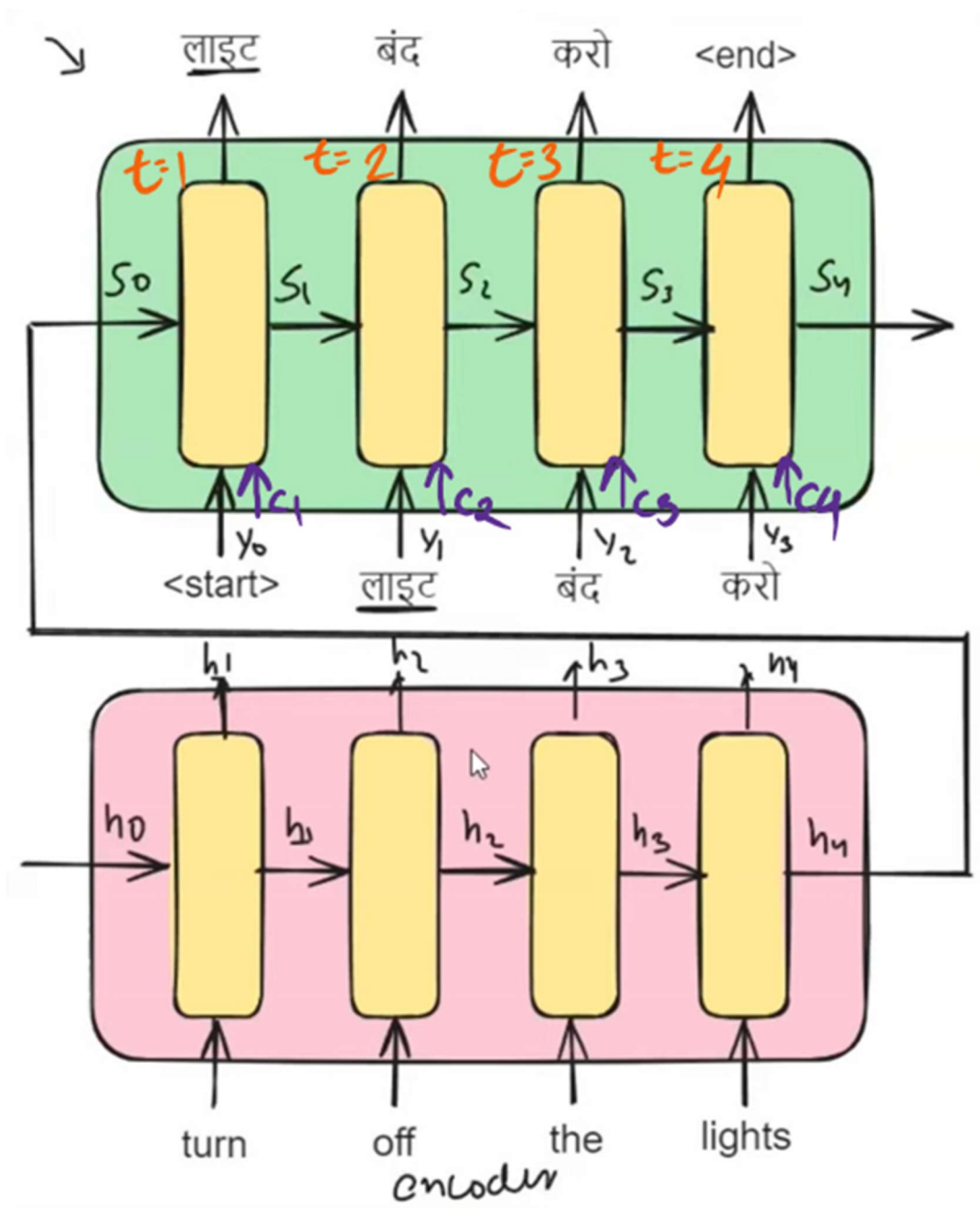
Now what is this attention mechanism

It basically means that for the decoder to print any word as the output, we tell the decoder that to print a particular word, these are the words or set of words from the input which will contribute the most (basically giving attention to the words)

Notations:

The internal hidden state of the encoder will be represented as h_0, h_1, h_2 etc

The internal hidden state of the decoder will be represented as s_0, s_1, s_2 etc



Modifying the encoder decoder architecture to make into attention architecture

In the encoder decoder architecture, for the decoder to print the output at any timestamp, lets say at $t = 2$, it takes 2 inputs s_1 and y_1

But in the attention architecture it takes 3 inputs s_1 , y_1 and one more piece of information that tells which internal state (h_1, h_2, h_3 etc) or a combination of internal states is more important to display the output word for the given timestamp by the decoder.

Let this one more piece of information be denoted by c_1, c_2, c_3 , and c_4

And this c_i is called as the attention input

- For a **encoder decoder architecture**, for the decoder to print the word at timestamp i , it requires **2 inputs** : $Y(i-1)$ and $S(i-1)$
- For an **attention architecture**, to print the word at timestamp i , it requires **3 inputs** : $Y(i-1)$, $S(i-1)$ and C_i

Now what exactly is this C_i (Attention input)

Now this C_i can either be a vector, or a scalar, or a matrix.

The significance of C_i is to tell the decoder that at the current timestamp, which hidden states of the encoders are useful.

This C_i is a vector because this will contain the information about the hidden states of the encoder

Now what will the dimension of C_i be?

Because sometimes it may happen that for the output word from the decoder, it requires just a single hidden state of the encoder, and sometimes combination of multiple hidden states of the encoder.

Example: In the output over here, the output is “light band karo”, the word “light (hindi)” just needed the word “light (english)” to do the translation, and the word “band (hindi)” just needed the set of words “turn off (english)” for translation

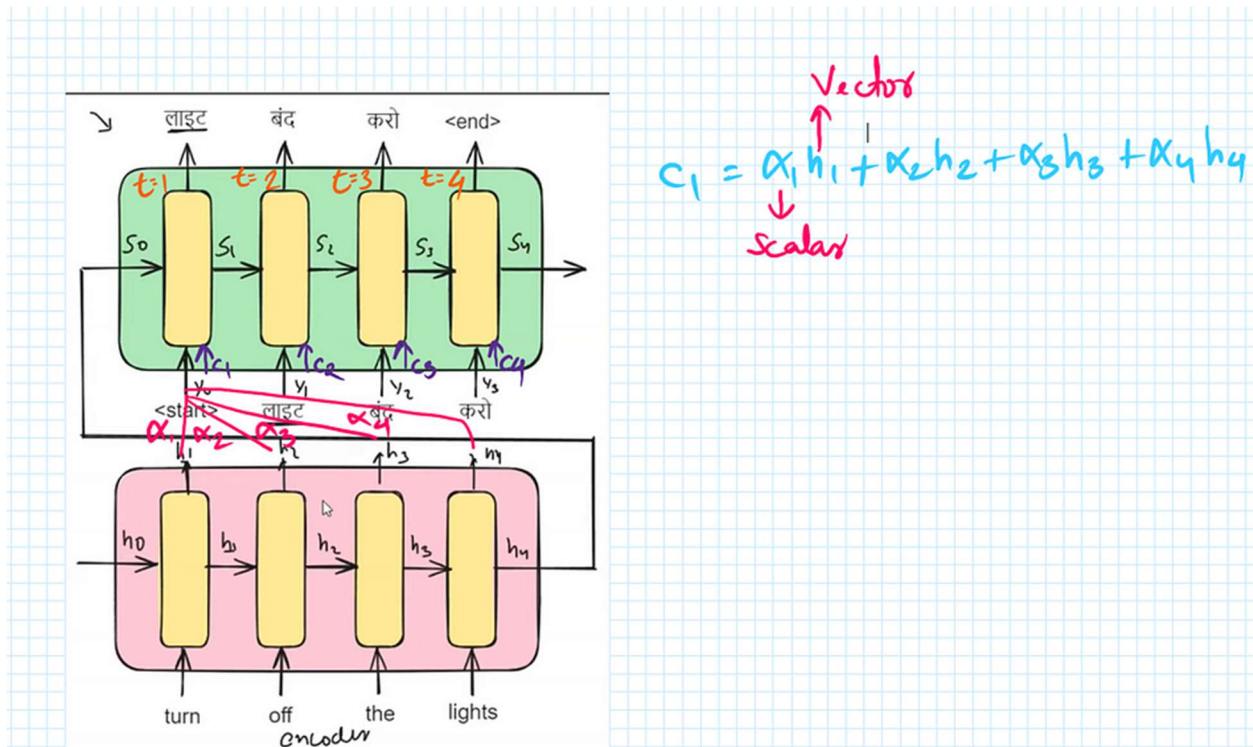
The answer is its dimension will be the same as that of h_i

Lets just assume that the dimension of the hidden states of the encoder is 4. Then the dimension of C_i will also be 4

So if we want send combination of multiple hidden states of the encoder to C_i , then we will perform weighted sum on it and then send it to C_i as to basically tell C_i , that for the current output, these are the significant words that you need to focus on more.

For any particular timestamp in the decoder, attention mechanism assigns weights to all the hidden states of the encoder.

Which means it will assign weight lets say α_1 to hidden state h_1 , α_2 to hidden state h_2 and so on and so forth.

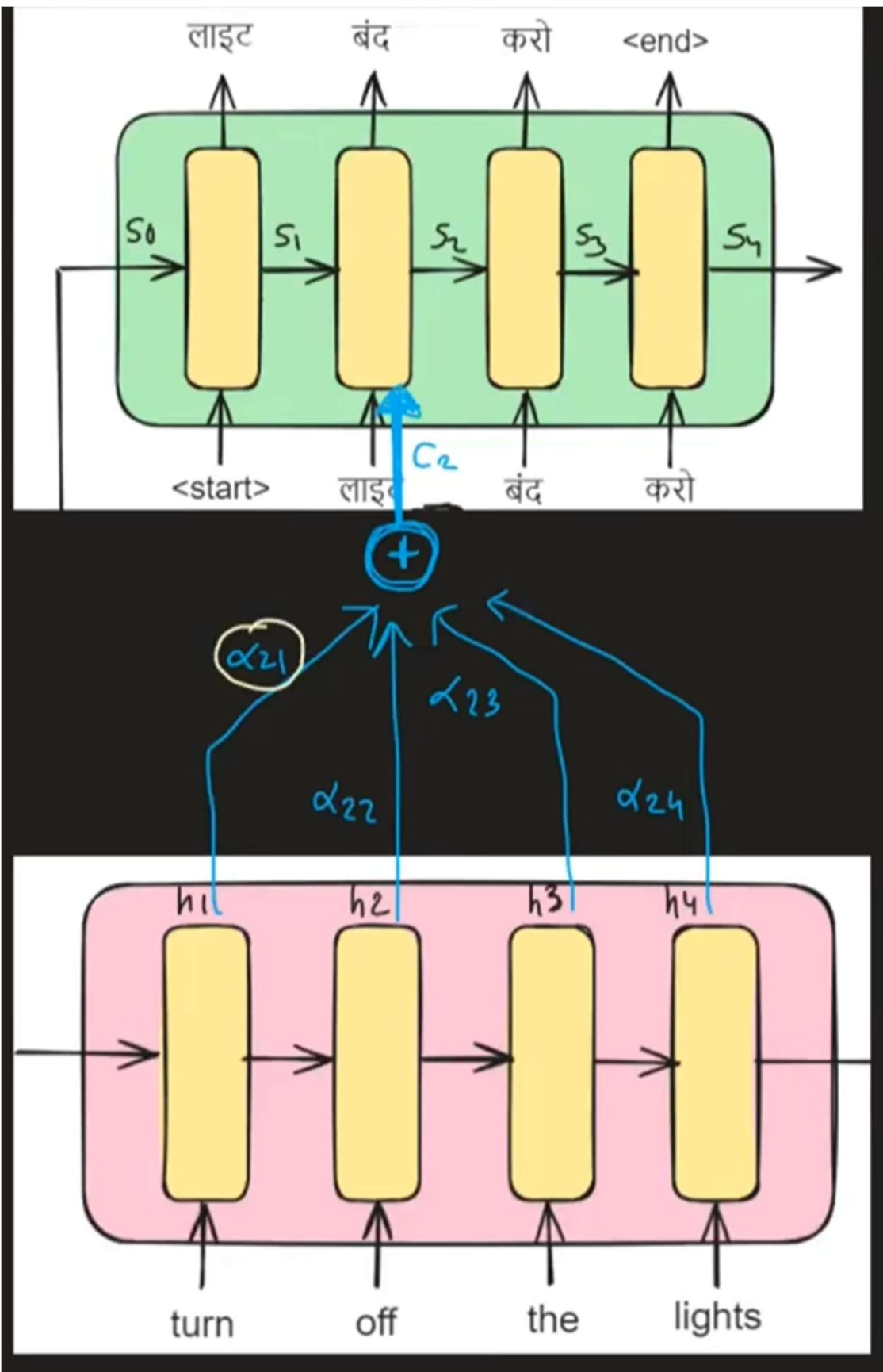


Modified encoder decoder architecture to make attention architecture and Equation for attention input

So now, depending on which value among $\alpha_1, \alpha_2, \alpha_3, \alpha_4$ is more, that hidden state will dominate more to produce the output

This same thing will happen for the next timestamp in the decoder. Again the same type of equation will be made for the next timestamp

Now, how is this alpha created



Modified encoder decoder architecture to make attention architecture

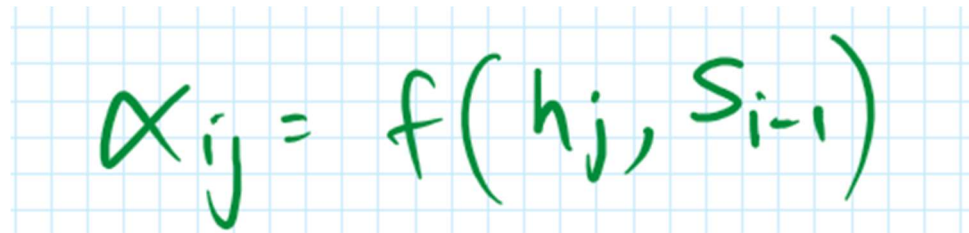
α_{21} here means how much importance will be given to the encoders at encoders timestamp $t = 1$ to print the output for timestamp $t = 2$ in the decoder

α_{21} is called as the alignment score or we can call this similarity score as well

α_{21} will depend on the following factors:

1. h_1 (hidden state of the encoder)
2. S_1 (previous hidden state of the decoder). Because given how much translation has been performed till now, how much significance the word "turn" or the hidden state h_1 has, to print the word at decoder for timestamp $t = 2$

So basically α_{21} will be a function of 2 quantities $f(h_1, S_1)$, and in general we could say

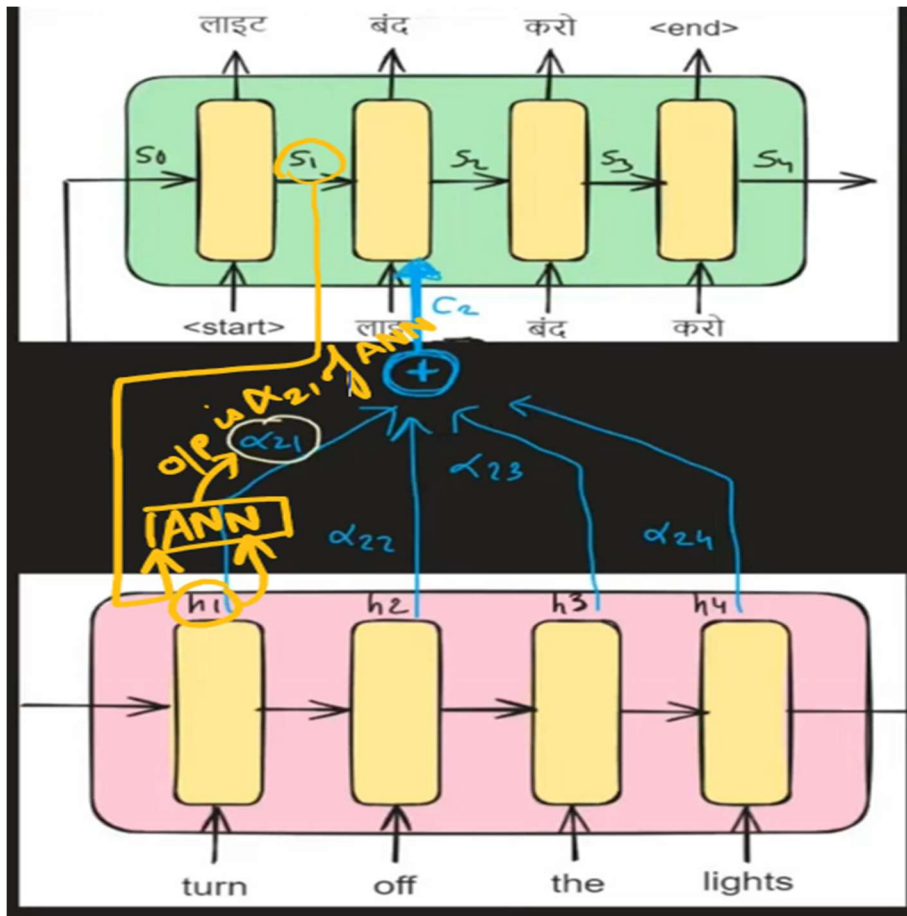

$$\alpha_{ij} = f(h_j, s_{i-1})$$

So now, the question arises, what will be this mathematical function

If this question would have been given to us, then we would have used different mathematical functions and checked their performance and the one which performed better would have been our mathematical function

But researchers thought why scratch our brains finding this mathematical function, we can directly leverage the use of ANN as **ANNs are universal function approximators — which means given enough data, we can make ANNs an approximator of any function**

ANN is a part of the decoder



Modified encoder decoder architecture to make attention architecture

h_1 and s_1 are passed as input to the ANN and the output which we get from this ANN is α_{21} .

This ANN will be trained along with the whole architecture

To give a better understanding of the alpha values, check the diagram below

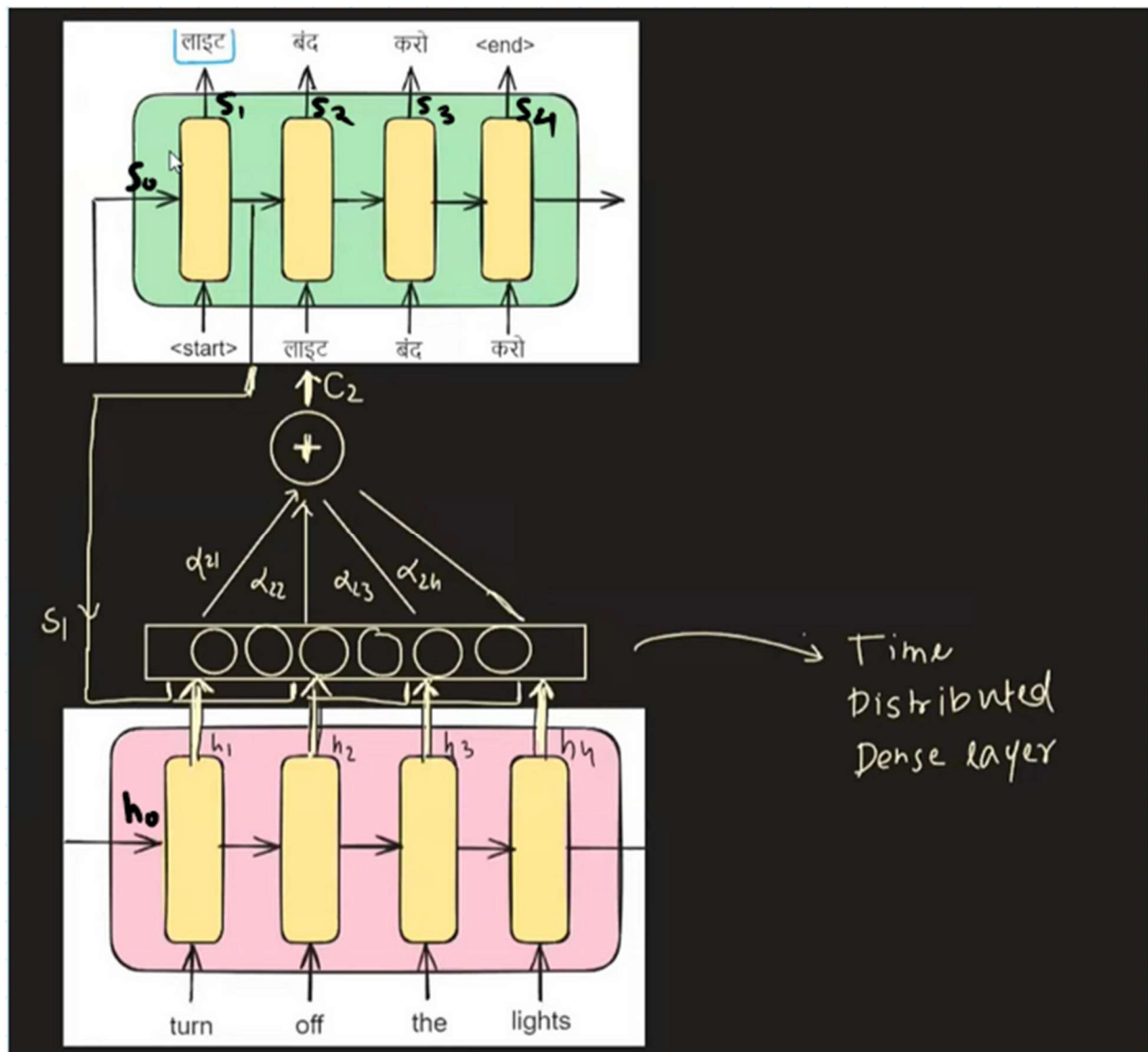
Press enter or click to view image in full size

लाइट	Turn α_{11}	off α_{12}	the α_{13}	light α_{14}
बंद	α_{21}	α_{22}	α_{23}	α_{24}
करो	α_{31}	α_{32}	α_{33}	α_{34}
<end>	α_{41}	α_{42}	α_{43}	α_{44}

Here α_{11} means the weight or the similarity score between "light(hindi)" and "turn(english)"

Here α_{32} means the weight or the similarity score between "karo(hindi)" and "off(english)" and so on and so forth

Complete architecture



Bahdanau Attention architecture

1. The input will be sent to the encoder on token by token level
2. And after each timestamp of the encoder, we will get $h_1, h_2, h_3, \dots$
3. At $t = 1$ in the decoder, we need to pass c_1 which is nothing but weighted sum of $\alpha_{11} \cdot h_1 + \alpha_{12} \cdot h_2 + \alpha_{13} \cdot h_3 + \alpha_{14} \cdot h_4$
4. Now we need to find these $\alpha_{11}, \alpha_{12}, \alpha_{13}, \alpha_{14}$, and we will find these values with the help of ANN

5. This alpha depends on the decoder's previous timestamp hidden state and encoders current timestamp. For example at $t = 1$, alpha depends on S_0 and h_1 , at $t = 2$, alpha depends on S_1 and h_2 etc
6. So, the input to the ANN will be the values of $S(i-1)$ and $h(j)$ (where i is the timestamp for decoder, and j is the timestamp for encoder)
7. We need to concatenate these 2 values and then send it to the ANN as input
8. Lets assume the dimension of $h_0, h_1, h_2, h_3, \dots$ be 1×4 . So now the dimension of $S_0, S_1, S_2, S_3, \dots$ will also be 1×4 . So now the concatenated value of $S(i-1)$ and $h(j)$ will be (4×8) . Assuming our ANN just contains 1 hidden layer which contains 3 neurons in it and 1 neuron in the next layer
9. Now this (4×8) concatenated values can be sent one row at a time to ANN or it can be sent completely as batch of 32.

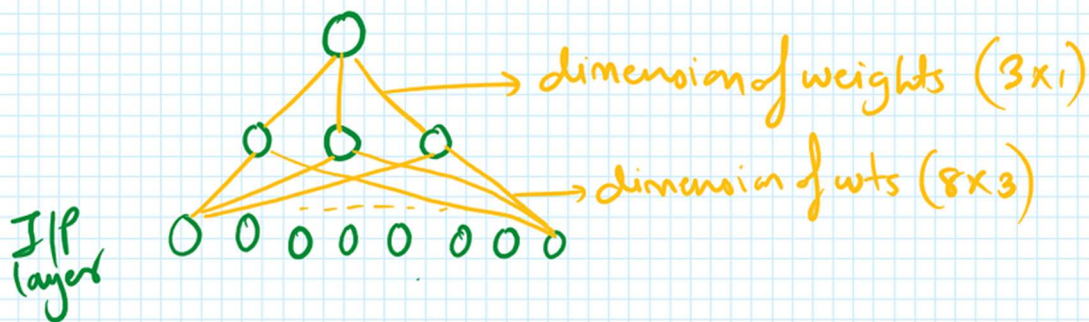
Press enter or click to view image in full size

$$\begin{aligned}
 S_0 &= [1 \times 4] \Rightarrow \text{let the values be } [e, f, g, h] \\
 h_1 &= [1 \times 4] \Rightarrow \text{let the values be } [a_1, b_1, c_1, d_1] \\
 h_2 &= [1 \times 4] \Rightarrow \text{let the values be } [a_2, b_2, c_2, d_2] \\
 h_3 &= [1 \times 4] \Rightarrow \text{let the values be } [a_3, b_3, c_3, d_3] \\
 h_4 &= [1 \times 4] \Rightarrow \text{let the values be } [a_4, b_4, c_4, d_4]
 \end{aligned}$$

Concatenated values of S_i and h_j

$$4 \times 8 \left\{ \begin{bmatrix} e, f, g, h, a_1, b_1, c_1, d_1 \\ e, f, g, h, a_2, b_2, c_2, d_2 \\ e, f, g, h, a_3, b_3, c_3, d_3 \\ e, f, g, h, a_4, b_4, c_4, d_4 \end{bmatrix} \right.$$

ANN



If to the above ANN, we send the whole (4×8) matrix, at the output we will get (4×1) vector

if we send one row at a time from (4×8) matrix then we need to club the output from the ANN together. (4 times)

Press enter or click to view image in full size

Now, let's assume that at the output of the ANN, we get the values o_1, o_2, o_3, o_4 .

These values are not normalized.

We will normalize these values by using softmax

$$\text{So } o_1 = \frac{o_1}{o_1 + o_2 + o_3 + o_4}$$

$$o_2 = \frac{o_2}{o_1 + o_2 + o_3 + o_4} \quad \text{etc}$$

Now the values of o_1, o_2, o_3, o_4 will be the values of $x_{i1}, x_{i2}, x_{i3}, x_{i4}$ (i is timestamp for decoder)

10. So now putting these values for alpha in ($\alpha_{11} \cdot h_1 + \alpha_{12} \cdot h_2 + \alpha_{13} \cdot h_3 + \alpha_{14} \cdot h_4$), we can get the value of C_1

11. So now giving the values of S_0 , $y(t-1)$ and C_t to the decoder, we get the output from the decoder. As these values will be sent to a Feed Forward Neural Network (ANN), then its output will be sent to the softmax function and from that the decoder will give the predicted word

12. Backpropagation will be applied when the decoder is done printing the outputs. Then the predicted values of the decoder will be compared to the actual true values and the loss will be calculated using categorical cross entropy. The way of calculating loss in attention architecture is the same as calculating loss in encoder decoder architecture

13. After the loss is calculated, the weights will be updated and again the process will repeat

14. This attention mechanism is called as **Bahdanau attention**. Bahdanau Attention is also called as **Additive Attention**

Now as the weights of this ANN will be the same for each timestamp of the decoder, this ANN is called as **time distributed fully connected neural network**. This ANN is called as **alignment model** (this word has been used for ANN in research paper)

Luong Attention

The attention input (C_i) is a weighted sum of hidden states of the encoder

$$C_i = \sum \alpha_{ij} h_j$$

And the values of alpha depends on 2 things, previous hidden state of the decoder (S_{i-1}) and hidden state of the encoder for the same timestamp (h_j)

$$\alpha_{ij} = f(S_{i-1}, h_j)$$

Value of alpha for bahdanau attention

But in Luong attention, alpha depends on the **current internal state of the decoder** and **not the previous internal state of the decoder**. The reason we used the current internal state of the decoder in Luong attention is so that we will have updated information in comparison to if we have used the previous internal state of the decoder, and also it allows to dynamically adjust the output of the decoder

Also, in Bahdanau attention, the mathematical function used to calculate the values of alpha is an ANN, but in Luong attention, it is just the dot product of S_i and h_j (Assuming the dimension of both S_i and h_j is same).

$$\alpha_{ij} = f(S_i, h_j)$$

Value of alpha for Luong attention

[Press enter or click to view image in full size](#)

If $S_i = \begin{bmatrix} a & b & c & d \end{bmatrix}$
 $h_j = \begin{bmatrix} e & f & g & h \end{bmatrix}$
then $S_i \cdot h_j = ae + bf + cg + dh$
Similarly we will get 4 values, then we will normalize these 4 values using softmax and those will be the values of our α_{ij}

Using ANN increases the parameters which slows down the training process.

So Luong attention used a simpler process to calculate the values of alpha.

They used dot product because if 2 vectors are similar their dot product will be more, and if 2 vectors are very different their dot product will be very less. So automatically the values of alpha will be decided depending on how much the current word in the decoder is related to the words in the encoder.

After the dot product is done, then these values will be sent to the softmax function for better interpretability of these values

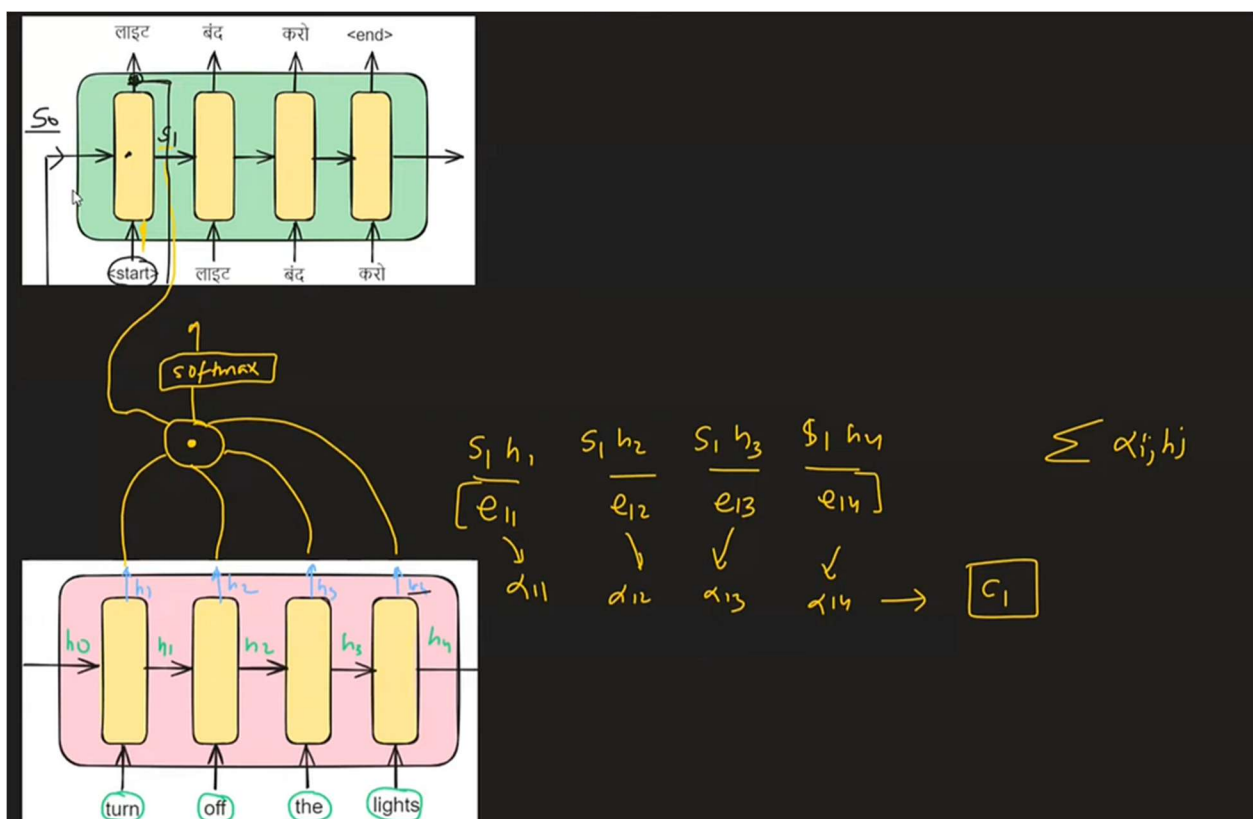
And this dot product is also telling us which hidden states of the encoder are more useful for printing the word at the current timestamp in the decoder

Luong attention gave better results compared to Bahdanau Attention

Also in Luong attention, the attention input C_i is given to the output of the LSTM cell whereas in Bahdanau it was given as input to the LSTM cell

Then once the attention input (C_i) is passed on to the output of the LSTM cell (h_i), these 2 vectors will be concatenated, then passed to a feed forward neural network and then softmax will be applied to give the output

Press enter or click to view image in full size



Luong Attention Architecture

This attention mechanism architecture was able to perform well when longer sentences were passed as input to it